

Lecture 11: language memory model

compiler optimizations, compiler barriers, language memory model, strict consistency, threads cannot be implemented as a library, visibility, volatile

Alexander Filatov
filatovaur@gmail.com

<https://github.com/Svazars/parallel-programming/blob/main/slides/pdf/l11.pdf>

In previous episodes: homework

"Is Parallel Programming Hard, And, If So, What Can You Do About It" (perfbok)

Appendix B "Why Memory Barriers?"

- section B.1 "Cache Structure". Be ready to draw and explain what is associative hardware cache.
- section B.2.4 "MESI Protocol Example"

In previous episodes

Abstractions and algorithms (high level)

- Race conditions, deadlocks, missed signals, ABA problem
- Events, Precedence, Consistency
- Progress conditions: wait-free, lock-free, obstruction-free, starvation-free, deadlock-free
- Read-Modify-Write operations

Programmers need at least: **Linearizability** (composability of different modules)

Hardware (low level)

- Cache coherency protocol: states, transitions, message passing
- Store buffering, Load buffering, Invalidate Queues, Interconnect topology
- Reorderings of memory operations on independent memory locations
- Weak (relaxed) consistency

Hardware provides at most: **Coherence** (linear order of writes for particular location)

The Gap

The Gap



The Gap



The Gap



Bridging the gap

High-level concurrent programming focuses on design-level problems:

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)
- avoiding logic errors (race condition, unsynchronized data access, resource leaks)

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)
- avoiding logic errors (race condition, unsynchronized data access, resource leaks)
- user-visible metrics (responsiveness, CPU utilization, memory consumption)

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)
- avoiding logic errors (race condition, unsynchronized data access, resource leaks)
- user-visible metrics (responsiveness, CPU utilization, memory consumption)
- error recovery, logging, graceful degradation

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)
- avoiding logic errors (race condition, unsynchronized data access, resource leaks)
- user-visible metrics (responsiveness, CPU utilization, memory consumption)
- error recovery, logging, graceful degradation

Low-level concurrent programming focuses on performance and location-level consistency:

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)
- avoiding logic errors (race condition, unsynchronized data access, resource leaks)
- user-visible metrics (responsiveness, CPU utilization, memory consumption)
- error recovery, logging, graceful degradation

Low-level concurrent programming focuses on performance and location-level consistency:

- scalability under contention

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)
- avoiding logic errors (race condition, unsynchronized data access, resource leaks)
- user-visible metrics (responsiveness, CPU utilization, memory consumption)
- error recovery, logging, graceful degradation

Low-level concurrent programming focuses on performance and location-level consistency:

- scalability under contention
- effective usage of scheduling quantum

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)
- avoiding logic errors (race condition, unsynchronized data access, resource leaks)
- user-visible metrics (responsiveness, CPU utilization, memory consumption)
- error recovery, logging, graceful degradation

Low-level concurrent programming focuses on performance and location-level consistency:

- scalability under contention
- effective usage of scheduling quantum
- avoiding false sharing, minimizing true sharing

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)
- avoiding logic errors (race condition, unsynchronized data access, resource leaks)
- user-visible metrics (responsiveness, CPU utilization, memory consumption)
- error recovery, logging, graceful degradation

Low-level concurrent programming focuses on performance and location-level consistency:

- scalability under contention
- effective usage of scheduling quantum
- avoiding false sharing, minimizing true sharing
- replication-friendly data structures

Bridging the gap

High-level concurrent programming focuses on design-level problems:

- separating function/computation and carrier/executor (Future, Executor)
- providing contracts and interfaces rather than particular implementations
- composability of concurrent modules (deadlock avoidance, restartable transactions)
- avoiding logic errors (race condition, unsynchronized data access, resource leaks)
- user-visible metrics (responsiveness, CPU utilization, memory consumption)
- error recovery, logging, graceful degradation

Low-level concurrent programming focuses on performance and location-level consistency:

- scalability under contention
- effective usage of scheduling quantum
- avoiding false sharing, minimizing true sharing
- replication-friendly data structures
- appropriate and lightweight memory barriers

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language.

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

- avoid most of concurrency pitfalls via source-level constraints

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

- avoid most of concurrency pitfalls via source-level constraints (expressive power?)

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

- avoid most of concurrency pitfalls via source-level constraints (expressive power?)
- simplify writing safe concurrent programs

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

- avoid most of concurrency pitfalls via source-level constraints (expressive power?)
- simplify writing safe concurrent programs (performance?)

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

- avoid most of concurrency pitfalls via source-level constraints (expressive power?)
- simplify writing safe concurrent programs (performance?)
- allow to write hardware-agnostic and high-performant concurrent programs

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

- avoid most of concurrency pitfalls via source-level constraints (expressive power?)
- simplify writing safe concurrent programs (performance?)
- allow to write hardware-agnostic and high-performant concurrent programs (bugs?)

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

- avoid most of concurrency pitfalls via source-level constraints (expressive power?)
- simplify writing safe concurrent programs (performance?)
- allow to write hardware-agnostic and high-performant concurrent programs (bugs?)
- were not designed for modern concurrency

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

- avoid most of concurrency pitfalls via source-level constraints (expressive power?)
- simplify writing safe concurrent programs (performance?)
- allow to write hardware-agnostic and high-performant concurrent programs (bugs?)
- were not designed for modern concurrency (single-threaded?)

Bridging the gap

- High level concurrent programming focuses on design-level problems
- Low level concurrent programming focuses on performance and location-level consistency
- Any programming language need to balance between low-level and high-level

There is no *perfect* programming language. But some languages

- avoid most of concurrency pitfalls via source-level constraints (expressive power?)
- simplify writing safe concurrent programs (performance?)
- allow to write hardware-agnostic and high-performant concurrent programs (bugs?)
- were not designed for modern concurrency (single-threaded?)

As a professional, you should be able to use any tool, not only "the favourite" one.

Supplementary materials

Unconditional benefit

- "Threads Cannot be Implemented as a Library" by Hans-J. Boehm¹
<https://courses.cs.washington.edu/courses/cse590p/05au/HPL-2004-209.pdf>
- "Memory Models" series by Russ Cox <https://research.swtch.com/mm>
- Herb Sutter, C++ and Beyond 2012, "Atomic Weapons" series
<https://youtu.be/A8eCG0qgvH4>
- Роман Елизаров, "Многопоточное программирование — теория и практика"
<https://youtu.be/mf4lC6Tpc1M>

Advanced material

- "Using JDK 9 Memory Order Modes" by Doug Lea
<https://gee.cs.oswego.edu/dl/html/j9mm.html>
- Aleksey Shipilëv, JMM series <https://shipilev.net>

¹ https://www.hboehm.info/misc_slides/pldi05_threads.pdf

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 Language memory models
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 Language memory models
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Inventing reads

```
static int a;  
void foo_1() {  
    while (true) {  
        int tmp = a;  
        if (tmp == 0) break;  
        do_something_with(tmp);  
    }  
}
```

Inventing reads

```
static int a;  
void foo_1() {  
    while (true) {  
        int tmp = a;  
        if (tmp == 0) break;  
        do_something_with(tmp);  
    }  
}
```

Could compiler use less registers for intermediate operations?

Inventing reads

```
static int a;
void foo_1() {
    while (true) {
        int tmp = a;
        if (tmp == 0) break;
        do_something_with(tmp);
    }
}
```

```
static int a;
void foo_2() {
    while (true) {

        if (a == 0) break;
        do_something_with(a);
    }
}
```

Could compiler use less registers for intermediate operations?

Removing reads

```
static int a;  
void foo_1() {  
    while (true) {  
        int tmp = a;  
        if (tmp == 0) break;  
        do_something_with(tmp);  
    }  
}
```

Removing reads

```
static int a;  
void foo_1() {  
    while (true) {  
        int tmp = a;  
        if (tmp == 0) break;  
        do_something_with(tmp);  
    }  
}
```

Could compiler avoid repeated memory loads?

Removing reads

```
static int a;
void foo_1() {
    while (true) {
        int tmp = a;
        if (tmp == 0) break;
        do_something_with(tmp);
    }
}
```

```
static int a;
void foo_2() {
    int tmp = a;
    if (tmp != 0)
        while (true) {
            do_something_with(tmp);
        }
}
```

Could compiler avoid repeated memory loads?

Removing reads

```
static int a;
void foo_1() {
    while (true) {
        int tmp = a;
        if (tmp == 0) break;
        do_something_with(tmp);
    }
}
```

x86-64 clang 16.0.0 -O2²

x86-64 gcc 13.1 -O2³

```
foo_1:
    push    rbx
    mov     ebx, [a]
    test    ebx, ebx
    je      .LBB1_2
.LBB1_1:
    mov     edi, ebx
    call    do_something_with
    jmp     .LBB1_1
.LBB1_2:
    pop     rbx
    ret
```

²<https://godbolt.org/z/99j3erzaE>

³<https://godbolt.org/z/fxzGEo1qf>

Load hoisting

```
static int a;  
void foo_1(bool c, int r1) {  
    if (c) {  
        r1 = a;  
    }  
}
```

```
static int a;  
void foo_2(bool c, int r1) {  
    int t = a;  
    r1 = c ? t : r1;  
}
```

CSE over lock

```
static int a;  
void foo_1(bool c, int r1, int r2) {  
    r1 = a;  
    lock();  
    r2 = a;  
}
```

```
static int a;  
void foo_2(bool c, int r1, int r2) {  
    r1 = a;  
    lock();  
    r2 = r1;  
}
```

Load hoisting + CSE over lock

```
static int a;
void foo_1() {
    if (c) {
        r1 = a;
    }
    lock();
    r2 = a;
}
```

```
static int a;
void foo_2() {
    int t = a;
    r1 = c ? t : r1;
    lock();
    r2 = a;
}
```

```
static int a;
void foo_3() {
    int t = a;
    r1 = c ? t : r1;
    lock();
    r2 = t;
}
```

Load hoisting + CSE over lock

```
static int a;
void foo_1() {
    if (c) {
        r1 = a;
    }
    lock();
    r2 = a;
}
```

```
static int a;
void foo_2() {
    int t = a;
    r1 = c ? t : r1;
    lock();
    r2 = a;
}
```

```
static int a;
void foo_3() {
    int t = a;
    r1 = c ? t : r1;
    lock();
    r2 = t;
}
```

When `c == false`, `a` is moved out of the critical region!⁴

⁴ <https://people.mpi-sws.org/~viktor/slides/2017-09-concur.pdf>

Compiler: friend or foe?

Various optimizations of plain memory accesses

Compiler: friend or foe?

Various optimizations of plain memory accesses

- invent reads or writes

Compiler: friend or foe?

Various optimizations of plain memory accesses

- invent reads or writes
- remove reads or writes

Compiler: friend or foe?

Various optimizations of plain memory accesses

- invent reads or writes
- remove reads or writes
- reorder memory operations

Compiler: friend or foe?

Various optimizations of plain memory accesses

- invent reads or writes
- remove reads or writes
- reorder memory operations

make concurrent reasoning almost impossible

Compiler: friend or foe?

Various optimizations of plain memory accesses

- invent reads or writes
- remove reads or writes
- reorder memory operations

make concurrent reasoning almost impossible

- no **linearizability**

Compiler: friend or foe?

Various optimizations of plain memory accesses

- invent reads or writes
- remove reads or writes
- reorder memory operations

make concurrent reasoning almost impossible

- no **linearizability**
- no **coherence**

Compiler: friend or foe?

Various optimizations of plain memory accesses

- invent reads or writes
- remove reads or writes
- reorder memory operations

make concurrent reasoning almost impossible

- no **linearizability**
- no **coherence**
- no **eventual visibility** (a.k.a. **progress**)

Taming compiler optimizations

Compiler reorders language-level constructs

Taming compiler optimizations

Compiler reorders language-level constructs

- to improve performance

Taming compiler optimizations

Compiler reorders language-level constructs

- to improve performance
- but keep consistent single-threaded behaviour

Taming compiler optimizations

Compiler reorders language-level constructs

- to improve performance
- but keep consistent single-threaded behaviour

We need to inform compiler that some invariants are important for multi-threaded execution:

Taming compiler optimizations

Compiler reorders language-level constructs

- to improve performance
- but keep consistent single-threaded behaviour

We need to inform compiler that some invariants are important for multi-threaded execution:

- do not reorder **this** and **that** operation (low-level consistency)

Taming compiler optimizations

Compiler reorders language-level constructs

- to improve performance
- but keep consistent single-threaded behaviour

We need to inform compiler that some invariants are important for multi-threaded execution:

- do not reorder **this** and **that** operation (low-level consistency)
- do not move operations out of critical sections (high-level consistency)

Taming compiler optimizations

Compiler reorders language-level constructs

- to improve performance
- but keep consistent single-threaded behaviour

We need to inform compiler that some invariants are important for multi-threaded execution:

- do not reorder **this** and **that** operation (low-level consistency)
- do not move operations out of critical sections (high-level consistency)
- do not invent new operations

Taming compiler optimizations

Compiler reorders language-level constructs

- to improve performance
- but keep consistent single-threaded behaviour

We need to inform compiler that some invariants are important for multi-threaded execution:

- do not reorder **this** and **that** operation (low-level consistency)
- do not move operations out of critical sections (high-level consistency)
- do not invent new operations
- do not remove or merge some operations

Taming compiler optimizations

Compiler reorders language-level constructs

- to improve performance
- but keep consistent single-threaded behaviour

We need to inform compiler that some invariants are important for multi-threaded execution:

- do not reorder **this** and **that** operation (low-level consistency)
- do not move operations out of critical sections (high-level consistency)
- do not invent new operations
- do not remove or merge some operations

We have already encountered such problems with reorderings of memory effects, remember?

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 Language memory models
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Barriers taxonomy: here we go again

Simplified taxonomy of barriers⁵:

- Store_Store, Store_Load, Load_Store, Load_Load

```
int x = static.data1;  
Store_Store();  
Store_Load();  
int y = static.data2;  
static.data3 = 17;
```

```
int x = static.data1;  
Load_Load();  
  
int y = static.data2;  
static.data3 = 17;
```

⁵ Could be misleading <https://shipilev.net/blog/2016/close-encounters-of-jmm-kind/>

Compiler barriers

```
int x, y;  
void foo1() {  
    x = 1;  
    y = 2;  
    x = 3;  
}
```

```
int x, y;  
void foo2() {  
    x = 1;  
    barrier();  
    y = 2;  
    x = 3;  
}
```

Compiler barriers

```
int x, y;  
void foo1() {  
    x = 1;  
    y = 2;  
    x = 3;  
}
```

```
foo1:  
    mov [y], 2  
    mov [x], 3  
    ret
```

```
int x, y;  
void foo2() {  
    x = 1;  
    barrier();  
    y = 2;  
    x = 3;  
}
```

```
foo2:  
    mov [x], 1  
    mov [y], 2  
    mov [x], 3  
    ret
```

Compiler barriers

```
int x, y;
void foo1() {
    x = 1;
    y = 2;
    x = 3;
}
```

```
foo1:
    mov [y], 2
    mov [x], 3
    ret
```

```
int x, y;
void foo2() {
    x = 1;
    barrier();
    y = 2;
    x = 3;
}
```

```
foo2:
    mov [x], 1
    mov [y], 2
    mov [x], 3
    ret
```

Warning: avoid such low-level tricks in modern programming languages!⁶

⁶<https://preshing.com/20120625/memory-ordering-at-compile-time/>

Question time

Question:

- `mov [x], 1`
- `mov [y], 2`
- `mov [x], 3`

Name hardware-level optimizations that will "reorder" these memory operations.



Reorderings: everywhere

Hardware

Reorderings: everywhere

Hardware

- anything could be reordered with everything

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order
- single memory cell is **coherent**

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order
- single memory cell is **coherent**
- ordering could be enforced by memory barriers

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order
- single memory cell is **coherent**
- ordering could be enforced by memory barriers

Compiler

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order
- single memory cell is **coherent**
- ordering could be enforced by memory barriers

Compiler

- anything could be reordered with everything

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order
- single memory cell is **coherent**
- ordering could be enforced by memory barriers

Compiler

- anything could be reordered with everything
- optimizations do not violate single-threaded behaviour

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order
- single memory cell is **coherent**
- ordering could be enforced by memory barriers

Compiler

- anything could be reordered with everything
- optimizations do not violate single-threaded behaviour
- some language constructs are **special** (e.g. synchronized)

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order
- single memory cell is **coherent**
- ordering could be enforced by memory barriers

Compiler

- anything could be reordered with everything
- optimizations do not violate single-threaded behaviour
- some language constructs are **special** (e.g. synchronized)
- ordering could be enforced by compiler barriers

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order
- single memory cell is **coherent**
- ordering could be enforced by memory barriers

Compiler

- anything could be reordered with everything
- optimizations do not violate single-threaded behaviour
- some language constructs are **special** (e.g. synchronized)
- ordering could be enforced by compiler barriers

Looks like problem is already solved!

Reorderings: everywhere

Hardware

- anything could be reordered with everything
- current CPU will "emulate" execution of single-threaded program "as if" in program order
- single memory cell is **coherent**
- ordering could be enforced by memory barriers

Compiler

- anything could be reordered with everything
- optimizations do not violate single-threaded behaviour
- some language constructs are **special** (e.g. synchronized)
- ordering could be enforced by compiler barriers

Looks like problem is already solved!

Reasoning about memory locations/compiler optimizations is mundane, complex and fragile, shouldn't we use something better?

Question time

Question: Barriers enforce ordering of operations. But compiler could "invent" or "remove" operations which will break some concurrent invariants. What should we do?



Consistency of memory operations in concurrent environment: challenges

Programming language should be high-level enough and **should not** depend on

Consistency of memory operations in concurrent environment: challenges

Programming language should be high-level enough and **should not** depend on

- Platform (different OSes, various CPU architectures)

Consistency of memory operations in concurrent environment: challenges

Programming language should be high-level enough and **should not** depend on

- Platform (different OSes, various CPU architectures)
- Optimizer (compilers)

Consistency of memory operations in concurrent environment: challenges

Programming language should be high-level enough and **should not** depend on

- Platform (different OSes, various CPU architectures)
- Optimizer (compilers)
- Runtime (memory management, RTTI, synchronization)

Consistency of memory operations in concurrent environment: challenges

Programming language should be high-level enough and **should not** depend on

- Platform (different OSes, various CPU architectures)
- Optimizer (compilers)
- Runtime (memory management, RTTI, synchronization)
- Language version (maybe)

Consistency of memory operations in concurrent environment: challenges

Programming language should be high-level enough and **should not** depend on

- Platform (different OSes, various CPU architectures)
- Optimizer (compilers)
- Runtime (memory management, RTTI, synchronization)
- Language version (maybe)

Programming language should be low-level enough and **should** provide

Consistency of memory operations in concurrent environment: challenges

Programming language should be high-level enough and **should not** depend on

- Platform (different OSes, various CPU architectures)
- Optimizer (compilers)
- Runtime (memory management, RTTI, synchronization)
- Language version (maybe)

Programming language should be low-level enough and **should** provide

- Consistency guarantees for explicitly synchronized memory operations

Consistency of memory operations in concurrent environment: challenges

Programming language should be high-level enough and **should not** depend on

- Platform (different OSes, various CPU architectures)
- Optimizer (compilers)
- Runtime (memory management, RTTI, synchronization)
- Language version (maybe)

Programming language should be low-level enough and **should** provide

- Consistency guarantees for explicitly synchronized memory operations
- Reasonable rules of concurrent execution

Consistency of memory operations in concurrent environment: challenges

Programming language should be high-level enough and **should not** depend on

- Platform (different OSes, various CPU architectures)
- Optimizer (compilers)
- Runtime (memory management, RTTI, synchronization)
- Language version (maybe)

Programming language should be low-level enough and **should** provide

- Consistency guarantees for explicitly synchronized memory operations
- Reasonable rules of concurrent execution

We need language memory model

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 Language memory models**
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Language Memory Model

Language Memory Model

- How to formalize it?
 - Documentation in human language
 - Machine-readable format
 - Executable algorithm
 - Set of optimizer rules

Language Memory Model

- How to formalize it?
 - Documentation in human language
 - Machine-readable format
 - Executable algorithm
 - Set of optimizer rules
- Prefer strict rules or use weak models?

Language Memory Model

- How to formalize it?
 - Documentation in human language
 - Machine-readable format
 - Executable algorithm
 - Set of optimizer rules
- Prefer strict rules or use weak models?
- How to check consistency of memory model itself?

Language Memory Model

- How to formalize it?
 - Documentation in human language
 - Machine-readable format
 - Executable algorithm
 - Set of optimizer rules
- Prefer strict rules or use weak models?
- How to check consistency of memory model itself?
- How to guarantee that every program could be unambiguously described?

Language Memory Model

- How to formalize it?
 - Documentation in human language
 - Machine-readable format
 - Executable algorithm
 - Set of optimizer rules
- Prefer strict rules or use weak models?
- How to check consistency of memory model itself?
- How to guarantee that every program could be unambiguously described?
- How to explain the rules to language users?

Language Memory Model

- How to formalize it?
 - Documentation in human language
 - Machine-readable format
 - Executable algorithm
 - Set of optimizer rules
- Prefer strict rules or use weak models?
- How to check consistency of memory model itself?
- How to guarantee that every program could be unambiguously described?
- How to explain the rules to language users?

There is no *perfect* programming language.

Language Memory Model

- How to formalize it?
 - Documentation in human language
 - Machine-readable format
 - Executable algorithm
 - Set of optimizer rules
- Prefer strict rules or use weak models?
- How to check consistency of memory model itself?
- How to guarantee that every program could be unambiguously described?
- How to explain the rules to language users?

There is no *perfect* programming language.

Any language tries to be *performant, safe, user-friendly, stable* ...

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 Language memory models
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

No man, no problem

Concurrency is complicated only when we have non-trivial communication

- via shared memory locations
- via concurrent primitives

No man, no problem

Concurrency is complicated only when we have non-trivial communication

- via shared memory locations
- via concurrent primitives

Use plain and simple approaches

No man, no problem

Concurrency is complicated only when we have non-trivial communication

- via shared memory locations
- via concurrent primitives

Use plain and simple approaches

- immutable data structures

No man, no problem

Concurrency is complicated only when we have non-trivial communication

- via shared memory locations
- via concurrent primitives

Use plain and simple approaches

- immutable data structures
- declarative description of computation

No man, no problem

Concurrency is complicated only when we have non-trivial communication

- via shared memory locations
- via concurrent primitives

Use plain and simple approaches

- immutable data structures
- declarative description of computation

SQL

No man, no problem

Concurrency is complicated only when we have non-trivial communication

- via shared memory locations
- via concurrent primitives

Use plain and simple approaches

- immutable data structures
- declarative description of computation

SQL , Clojure

No man, no problem

Concurrency is complicated only when we have non-trivial communication

- via shared memory locations
- via concurrent primitives

Use plain and simple approaches

- immutable data structures
- declarative description of computation

SQL , Clojure , Haskell

No man, no problem

Concurrency is complicated only when we have non-trivial communication

- via shared memory locations
- via concurrent primitives

Use plain and simple approaches

- immutable data structures
- declarative description of computation

SQL , Clojure , Haskell

All told, a monad in X is just a monoid in the category of endofunctors of X , with product $\times$ replaced by composition of endofunctors and unit set by the identity endofunctor.

No man, no problem

Concurrency is complicated only when we have non-trivial communication

- via shared memory locations
- via concurrent primitives

Use plain and simple approaches

- immutable data structures
- declarative description of computation

SQL , Clojure , Haskell

All told, a monad in X is just a monoid in the category of endofunctors of X , with product $\times$ replaced by composition of endofunctors and unit set by the identity endofunctor.

Is it about *language* or *programming style*?

Immutability

Immutable data structure

Immutability

Immutable data structure

- different threads could simultaneously read

Immutability

Immutable data structure

- different threads could simultaneously read
- conflicts (data races) are impossible

Immutability

Immutable data structure

- different threads could simultaneously read
- conflicts (data races) are impossible

How to update such data structure when something happens?

Immutability

Immutable data structure

- different threads could simultaneously read
- conflicts (data races) are impossible

How to update such data structure when something happens?

- Create new immutable instance with up-to-date information

Immutability

Immutable data structure

- different threads could simultaneously read
- conflicts (data races) are impossible

How to update such data structure when something happens?

- Create new immutable instance with up-to-date information

Few problems

Immutability

Immutable data structure

- different threads could simultaneously read
- conflicts (data races) are impossible

How to update such data structure when something happens?

- Create new immutable instance with up-to-date information

Few problems

- Overheads for creation of large data structures

Immutability

Immutable data structure

- different threads could simultaneously read
- conflicts (data races) are impossible

How to update such data structure when something happens?

- Create new immutable instance with up-to-date information

Few problems

- Overheads for creation of large data structures
- "Publish" data structure for all threads == write to shared memory location

Declarative DSL

```
java.util.stream7  
  
int sumOfWeights = widgets.parallelStream()  
    .filter(b -> b.getColor() == RED)  
    .mapToInt(b -> b.getWeight())  
    .sum();
```

⁷ <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/stream/package-summary.html>

Declarative DSL

```
java.util.stream7  
  
int sumOfWeights = widgets.parallelStream()  
    .filter(b -> b.getColor() == RED)  
    .mapToInt(b -> b.getWeight())  
    .sum();
```

Solve any embarrassingly parallel problem in few readable lines.

⁷ <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/stream/package-summary.html>

Declarative DSL

```
java.util.stream7  
  
int sumOfWeights = widgets.parallelStream()  
    .filter(b -> b.getColor() == RED)  
    .mapToInt(b -> b.getWeight())  
    .sum();
```

Solve any embarrassingly parallel problem in few readable lines.
All safety and efficiency will happen under the hood.

⁷ <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/stream/package-summary.html>

Declarative DSL

Solve any embarrassingly parallel problem in few readable lines.

Declarative DSL

Solve any embarrassingly parallel problem in few readable lines.

- Java parallel streams <https://docs.oracle.com/javase/tutorial/collections/streams/parallelism.html>
- OpenMP <https://www.openmp.org/>
- Intel TBB <https://github.com/oneapi-src/oneTBB>
- MPI <https://www.open-mpi.org/>
- MapReduce <https://research.google/pubs/pub62/>
- Resilient Distributed Datasets <https://dl.acm.org/doi/10.5555/2228298.2228301>

Declarative DSL

Solve any embarrassingly parallel problem in few readable lines.

- Java parallel streams <https://docs.oracle.com/javase/tutorial/collections/streams/parallelism.html>
- OpenMP <https://www.openmp.org/>
- Intel TBB <https://github.com/oneapi-src/oneTBB>
- MPI <https://www.open-mpi.org/>
- MapReduce <https://research.google/pubs/pub62/>
- Resilient Distributed Datasets <https://dl.acm.org/doi/10.5555/2228298.2228301>

First and obvious choice for any language and any application.

Declarative DSL

Solve any embarrassingly parallel problem in few readable lines.

- Java parallel streams <https://docs.oracle.com/javase/tutorial/collections/streams/parallelism.html>
- OpenMP <https://www.openmp.org/>
- Intel TBB <https://github.com/oneapi-src/oneTBB>
- MPI <https://www.open-mpi.org/>
- MapReduce <https://research.google/pubs/pub62/>
- Resilient Distributed Datasets <https://dl.acm.org/doi/10.5555/2228298.2228301>

First and obvious choice for any language and any application. Unless it does not fit.

Declarative DSL

Solve any embarrassingly parallel problem in few readable lines.

- Java parallel streams <https://docs.oracle.com/javase/tutorial/collections/streams/parallelism.html>
- OpenMP <https://www.openmp.org/>
- Intel TBB <https://github.com/oneapi-src/oneTBB>
- MPI <https://www.open-mpi.org/>
- MapReduce <https://research.google/pubs/pub62/>
- Resilient Distributed Datasets <https://dl.acm.org/doi/10.5555/2228298.2228301>

First and obvious choice for any language and any application. Unless it does not fit. Implementations of such DSL are concurrently writing to shared memory locations.

Actor model

Programming model with no shared state⁸.

⁸ https://en.wikipedia.org/wiki/Actor_model

Actor model

Programming model with no shared state⁸.

All computational agents (lightweight processes) are independent and communicate via message passing.

⁸https://en.wikipedia.org/wiki/Actor_model

Question time

Question: Name your favourite concurrent message-passing protocol



Actor model

Programming model with no shared state.

All computational agents (lightweight processes) are independent and communicate via message passing.

Actor model

Programming model with no shared state.

All computational agents (lightweight processes) are independent and communicate via message passing.

- Erlang⁹
- Akka actors¹⁰

⁹ <https://www.erlang.org/>

¹⁰ <https://doc.akka.io/docs/akka/current/typed/actors.html#akka-actors>

Actor model

Programming model with no shared state.

All computational agents (lightweight processes) are independent and communicate via message passing.

- Erlang⁹
- Akka actors¹⁰

Implementations of such systems are concurrently writing to shared memory locations.

⁹ <https://www.erlang.org/>

¹⁰ <https://doc.akka.io/docs/akka/current/typed/actors.html#akka-actors>

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 **Language memory models**
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - **Partial memory model: bad things should be avoided**
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Be in Sync or Die Tryin

Swift¹¹

Concurrent write/write or read/write access to the same location in memory generally remains undefined/illegal behavior, unless all such access is done through a special set of primitive atomic operations.

¹¹<https://github.com/apple/swift-evolution/blob/main/proposals/0282-atomics.md>

Be in Sync or Die Tryin

Swift¹¹

Concurrent write/write or read/write access to the same location in memory generally remains undefined/illegal behavior, unless all such access is done through a special set of primitive atomic operations.

¹¹<https://github.com/apple/swift-evolution/blob/main/proposals/0282-atomics.md>

Be in Sync or Die Tryin

Swift¹¹

Concurrent write/write access to the same location remains illegal behavior, unless is done through atomic operations.

¹¹<https://github.com/apple/swift-evolution/blob/main/proposals/0282-atomics.md>

Be in Sync or Die Tryin

Swift¹¹

Concurrent write/write access to the same location remains illegal behavior, unless is done through atomic operations.

```
import Foundation
class Bird {}
var S = Bird()
let q = DispatchQueue.global(qos: .default)
q.async { while(true) { S = Bird() } }
while(true) { S = Bird() }
```

¹¹ <https://github.com/apple/swift-evolution/blob/main/proposals/0282-atomics.md>

Be in Sync or Die Tryin

Swift¹¹

Concurrent write/write access to the same location remains illegal behavior, unless is done through atomic operations.

```
import Foundation
class Bird {}
var S = Bird()
let q = DispatchQueue.global(qos: .default)
q.async { while(true) { S = Bird() } }
while(true) { S = Bird() }
```

Concurrent writes are violating naive implementation of automatic reference counting^{12,13}.
Program crashes with double free or corruption.

¹¹<https://github.com/apple/swift-evolution/blob/main/proposals/0282-atomics.md>

¹²<https://tonygoold.github.io/arcempire/>

¹³<https://github.com/apple/swift/blob/main/docs/proposals/Concurrency.rst>

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 **Language memory models**
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - **Strict consistency: GIL**
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Mutex-based strict consistency

Strict consistency – all operations happens atomically and have total order¹⁴.

¹⁴ https://en.wikipedia.org/wiki/Consistency_model#Strict_consistency

Mutex-based strict consistency

Strict consistency – all operations happens atomically and have total order.

```
void thread1() {
```

```
    foo();
```

```
    bar();
```

```
}
```

```
void thread2() {
```

```
    baz();
```

```
    foo();
```

```
}
```

Mutex-based strict consistency

Strict consistency – all operations happens atomically and have total order.

```
void thread1() {  
    lock();  
    foo();  
    unlock();  
    lock();  
    bar();  
    unlock();  
}
```

```
void thread2() {  
    lock();  
    baz();  
    unlock();  
    lock();  
    foo();  
    unlock();  
}
```

Mutex-based strict consistency

Strict consistency – all operations happens atomically and have total order.

```
void thread1() {  
    GIL.lock();  
    foo();  
    GIL.unlock();  
    GIL.lock();  
    bar();  
    GIL.unlock();  
}
```

```
void thread2() {  
    GIL.lock();  
    baz();  
    GIL.unlock();  
    GIL.lock();  
    foo();  
    GIL.unlock();  
}
```

Global mutex guards every "instruction"

Mutex-based strict consistency

Strict consistency – all operations happens atomically and have total order.
Global Interpreter Lock (GIL)¹⁵ guards every "instruction"

¹⁵https://en.wikipedia.org/wiki/Global_interpreter_lock

Mutex-based strict consistency

Strict consistency – all operations happens atomically and have total order.

Global Interpreter Lock (GIL)¹⁵ guards every "instruction"

Language do have threads¹⁶, they are "concurrent but not parallel"

¹⁵ https://en.wikipedia.org/wiki/Global_interpreter_lock

¹⁶ <https://docs.python.org/3/library/threading.html>

Mutex-based strict consistency

Strict consistency – all operations happens atomically and have total order.

Global Interpreter Lock (GIL)¹⁵ guards every "instruction"

Language do have threads¹⁶, they are "concurrent but not parallel"

GIL is technical decision that may highly influence the whole ecosystem

- Alternative language implementations¹⁷
- Memory model¹⁸
- Interoperability with other languages¹⁹

¹⁵ https://en.wikipedia.org/wiki/Global_interpreter_lock

¹⁶ <https://docs.python.org/3/library/threading.html>

¹⁷ <https://doc.pypy.org/en/latest/faq.html#does-pypy-have-a-gil-why>

¹⁸ <https://peps.python.org/pep-0583/>

¹⁹ <https://peps.python.org/pep-0703/>

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 **Language memory models**
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - **Strict consistency: Event Loop**
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

- Forbid multithreading on language level

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

- Forbid multithreading on language level
- The only thread processes events, every event could create other (possibly delayed) events

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

- Forbid multithreading on language level
- The only thread processes events, every event could create other (possibly delayed) events
- JavaScript Event Loop²⁰

²⁰ https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

- Forbid multithreading on language level
- The only thread processes events, every event could create other (possibly delayed) events
- JavaScript Event Loop²⁰

Each job is processed completely before any other job is processed. This offers some nice properties when reasoning about your program, including the fact that whenever a function runs, it cannot be preempted and will run entirely before any other code runs (and can modify data the function manipulates).

²⁰ https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

- Forbid multithreading on language level
 - The only thread processes events, every event could create other (possibly delayed) events
 - JavaScript Event Loop
-

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

- Forbid multithreading on language level
- The only thread processes events, every event could create other (possibly delayed) events
- JavaScript Event Loop

If you need parallelism – use dedicated API²¹

²¹https://www.w3schools.com/html/html5_webworkers.asp

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

- Forbid multithreading on language level
- The only thread processes events, every event could create other (possibly delayed) events
- JavaScript Event Loop

If you need parallelism – use dedicated API²¹

- Safety – message passing with data copying²²

²¹https://www.w3schools.com/html/html5_webworkers.asp

²²https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Structured_clone_algorithm

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

- Forbid multithreading on language level
- The only thread processes events, every event could create other (possibly delayed) events
- JavaScript Event Loop

If you need parallelism – use dedicated API²¹

- Safety – message passing with data copying²²
- Performance – shared byte array²³ + atomics²⁴

²¹https://www.w3schools.com/html/html5_webworkers.asp

²²https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Structured_clone_algorithm

²³https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer

²⁴https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics

Single-threaded strict consistency

Strict consistency – all operations happen atomically and have total order.

- Forbid multithreading on language level
- The only thread processes events, every event could create other (possibly delayed) events
- JavaScript Event Loop

If you need parallelism – use dedicated API²¹

- Safety – message passing with data copying²²
- Performance – shared byte array²³ + atomics²⁴

As soon as you have concurrent access – you immediately encounter "serious" problems²⁵

²¹https://www.w3schools.com/html/html5_webworkers.asp

²²https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Structured_clone_algorithm

²³https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer

²⁴https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics

²⁵"Repairing and Mechanising the JavaScript Relaxed Memory Model" <https://arxiv.org/abs/2005.10554> 

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 **Language memory models**
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - **Partial memory model: threading as a service**
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Threads as a library

C/C++ before C++11²⁶

²⁶ https://en.wikipedia.org/wiki/C%2B%2B11#Multithreading_memory_model

Threads as a library

C/C++ before C++11²⁶

- There were no threads in language specification
 - There were libraries that implement threading
- Every project was free to invent custom designs, data structures, conventions
 - Compiler-specific
 - CPU-specific
 - OS-specific
- Any data race is undefined behaviour

²⁶ https://en.wikipedia.org/wiki/C%2B%2B11#Multithreading_memory_model

Threads as universal library

C/C++ before concurrent specifications + POSIX threads²⁷

²⁷ <https://en.wikipedia.org/wiki/Pthreads>

Threads as universal library

C/C++ before concurrent specifications + POSIX threads²⁷

- There were no threads in language specification
 - There was **universal** library to implement threading
- Portable solution
- Any data race is undefined behaviour. Use mutexes, Luke!

²⁷ <https://en.wikipedia.org/wiki/Pthreads>

Threads as universal library

C/C++ before concurrent specifications + POSIX threads²⁷

- There were no threads in language specification
 - There was **universal** library to implement threading
- Portable solution
- Any data race is undefined behaviour. Use mutexes, Luke!

Do not underestimate the complexity of implementing such library^{28,29}

²⁷ <https://en.wikipedia.org/wiki/Pthreads>

²⁸ <https://probablydance.com/2020/10/31/using-tla-in-the-real-world-to-understand-a-glibc-bug/>

²⁹ <https://probablydance.com/2022/09/17/finding-the-second-bug-in-glibcs-condition-variable/>

Threads as universal library

C/C++ before concurrent specifications + POSIX threads²⁷

- There were no threads in language specification
 - There was **universal** library to implement threading
- Portable solution
- Any data race is undefined behaviour. Use mutexes, Luke!

Do not underestimate the complexity of implementing such library^{28,29}

"Threads Cannot Be Implemented As a Library"³⁰

²⁷ <https://en.wikipedia.org/wiki/Pthreads>

²⁸ <https://probablydance.com/2020/10/31/using-tla-in-the-real-world-to-understand-a-glibc-bug/>

²⁹ <https://probablydance.com/2022/09/17/finding-the-second-bug-in-glibcs-condition-variable/>

³⁰ <https://www.hpl.hp.com/techreports/2004/HPL-2004-209.pdf>

Threads as universal library

C/C++ before concurrent specifications + POSIX threads²⁷

- There were no threads in language specification
 - There was **universal** library to implement threading
- Portable solution
- Any data race is undefined behaviour. Use mutexes, Luke!

Do not underestimate the complexity of implementing such library^{28,29}

"Threads Cannot Be Implemented As a Library"³⁰

The Pthreads specification prohibits races, i.e. accesses to a shared variable while another thread is modifying it. ... the problem here is that whether or not a race exists depends on the semantics of the programming language, which in turn requires that we have a properly defined memory model. Thus this definition is circular.

²⁷ <https://en.wikipedia.org/wiki/Pthreads>

²⁸ <https://probablydance.com/2020/10/31/using-tla-in-the-real-world-to-understand-a-glibc-bug/>

²⁹ <https://probablydance.com/2022/09/17/finding-the-second-bug-in-glibcs-condition-variable/>

³⁰ <https://www.hpl.hp.com/techreports/2004/HPL-2004-209.pdf>

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 Language memory models**
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset**
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Explicit concurrent operations

Specify subset of programming language that provides consistent multithreaded execution.

Explicit concurrent operations

Specify subset of programming language that provides consistent multithreaded execution.

- C/C++ atomics

Explicit concurrent operations

Specify subset of programming language that provides consistent multithreaded execution.

- C/C++ atomics
- Swift/ObjC NSLocking

Explicit concurrent operations

Specify subset of programming language that provides consistent multithreaded execution.

- C/C++ atomics
- Swift/ObjC NSLocking
- Java-1996 volatile

Explicit concurrent operations

Specify subset of programming language that provides consistent multithreaded execution.

- C/C++ atomics
- Swift/ObjC NSLocking
- Java-1996 volatile

Part of a **language**, not just a library!

Explicit concurrent operations

Specify subset of programming language that provides consistent multithreaded execution.

- C/C++ atomics
- Swift/ObjC NSLocking
- Java-1996 volatile

Part of a **language**, not just a library!

Not that easy to do: "The Java Memory Model is Fatally Flawed" , William Pugh, 2000³¹

³¹<http://www.cs.umd.edu/~pugh/java/broken.pdf>

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 Language memory models**
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - **Memory model: concurrency embedded into language**
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Formal language memory model

Mathematics to the max

Formal language memory model

Mathematics to the max

- An action a is described by a tuple $\langle t, k, v, u \rangle$ comprising ...

Formal language memory model

Mathematics to the max

- An action a is described by a tuple $\langle t, k, v, u \rangle$ comprising ...
- Partial and linear orders; transitive closure of binary relations; happens-before

Formal language memory model

Mathematics to the max

- An action a is described by a tuple $\langle t, k, v, u \rangle$ comprising ...
- Partial and linear orders; transitive closure of binary relations; happens-before
- Causality requirements, circular happens-before, out-of-thin-air problem

Formal language memory model

Mathematics to the max

- An action a is described by a tuple $\langle t, k, v, u \rangle$ comprising ...
- Partial and linear orders; transitive closure of binary relations; happens-before
- Causality requirements, circular happens-before, out-of-thin-air problem
- Adaptation to h/w models³²

³²"JSR-133 Cookbook for Compiler Writers" <https://gee.cs.oswego.edu/dl/jmm/cookbook.html>

Formal language memory model

Mathematics to the max

- An action a is described by a tuple $\langle t, k, v, u \rangle$ comprising ...
- Partial and linear orders; transitive closure of binary relations; happens-before
- Causality requirements, circular happens-before, out-of-thin-air problem
- Adaptation to h/w models³²
- Every data race is specified: allowed and forbidden outcomes

³²"JSR-133 Cookbook for Compiler Writers" <https://gee.cs.oswego.edu/dl/jmm/cookbook.html>

Formal language memory model

Mathematics to the max

- An action a is described by a tuple $\langle t, k, v, u \rangle$ comprising ...
- Partial and linear orders; transitive closure of binary relations; happens-before
- Causality requirements, circular happens-before, out-of-thin-air problem
- Adaptation to h/w models³²
- Every data race is specified: allowed and forbidden outcomes

Few problems

- Complicated, takes a lot of time, expensive to maintain
- Not ideal³³
- Hard to explain, hard to use in practice

³² "JSR-133 Cookbook for Compiler Writers" <https://gee.cs.oswego.edu/dl/jmm/cookbook.html>

³³ "Java Memory Model Examples: Good, Bad and Ugly" <https://groups.inf.ed.ac.uk/request/jmmexamples.pdf>

Formal Memory Models: useful?

Goals of our 1-semester introductory course to concurrency

Formal Memory Models: useful?

Goals of our 1-semester introductory course to concurrency

- Show how to write
 - correct
 - **understandable**
 - performant
- concurrent code

Formal Memory Models: useful?

Goals of our 1-semester introductory course to concurrency

- Show how to write
 - correct
 - **understandable**
 - performantconcurrent code
- Show **basic** concurrent concepts

Formal Memory Models: useful?

Goals of our 1-semester introductory course to concurrency

- Show how to write
 - correct
 - **understandable**
 - performantconcurrent code
- Show **basic** concurrent concepts
- Convince to use simple rather complicated approaches

Formal Memory Models: useful?

Goals of our 1-semester introductory course to concurrency

- Show how to write
 - correct
 - **understandable**
 - performant

concurrent code

- Show **basic** concurrent concepts
- Convince to use simple rather complicated approaches

Non-goals

Formal Memory Models: useful?

Goals of our 1-semester introductory course to concurrency

- Show how to write
 - correct
 - **understandable**
 - performant
- concurrent code
- Show **basic** concurrent concepts
- Convince to use simple rather complicated approaches

Non-goals

- Look smart and mathematically inclined

Formal Memory Models: useful?

Goals of our 1-semester introductory course to concurrency

- Show how to write
 - correct
 - **understandable**
 - performant
- concurrent code
- Show **basic** concurrent concepts
- Convince to use simple rather complicated approaches

Non-goals

- Look smart and mathematically inclined
- Study state-of-the art concurrency

Formal Memory Models: useful?

Goals of our 1-semester introductory course to concurrency

- Show how to write
 - correct
 - **understandable**
 - performant
- concurrent code
- Show **basic** concurrent concepts
- Convince to use simple rather complicated approaches

Non-goals

- Look smart and mathematically inclined
- Study state-of-the art concurrency
- Achieve top performance in our concurrent programs

Formal Memory Models: useful?

Goals of our 1-semester introductory course to concurrency

- Show how to write
 - correct
 - **understandable**
 - performant
- concurrent code
- Show **basic** concurrent concepts
- Convince to use simple rather complicated approaches

Non-goals

- Look smart and mathematically inclined
- Study state-of-the art concurrency
- Achieve top performance in our concurrent programs

Studying concurrent consistency, proving theorems on Lecture 7 and Lecture 8, analyzing cache coherence was enough.

Patterns

Doug Lea, private communication with Aleksey Shipilev, 2013³⁴

The best way to build up a small repertoire of constructions that you know the answers for and then never think about the JMM rules again unless you are forced to do so! Literally nobody likes figuring things out from the JMM rules as stated, or can even routinely do so correctly. This is one of the many reasons we need to overhaul JMM someday.

³⁴ Citation from <https://shipilev.net/blog/2014/jmm-pragmatics>, slide 109

Concurrent patterns

Could be found in books

- "Java Concurrency in Practice"
- "Effective Java"
- "The Art of Multiprocessor Programming"
- "Is Parallel Programming Hard, And, If So, What Can You Do About It?"

Could be found in documentation `java.util.concurrent`

We studied this

- Lock, code locking, data locking, lock splitting (Lecture 2)
- Executor, Backoff, Condition (Lecture 3)
- Monitor, CountdownLatch, ThreadLocal, replication (Lecture 4)
- Concurrent state machine, cancellation, interruption (Lecture 5)

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 Language memory models
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - **Visibility**
 - Volatile
- 5 Summary

Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = 42;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = 42;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

Do you see race condition that prevents threadB from printing integer?

Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = 42;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

Do you see race condition that prevents threadB from printing integer?
NullPointerException

Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = (1L << 40) + 1;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = (1L << 40) + 1;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

- Could program print 1?

Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = (1L << 40) + 1;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

- Could program print 1?
- Could program print 1099511627776 ($1 \ll 40$)?

Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = (1L << 40) + 1;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

- Could program print 1?
- Could program print 1099511627776 ($1 \ll 40$)?
- Could program print 1099511627777?

Question time

Question: Word tearing.

For the purposes of the Java programming language memory model, a single write to a non-volatile long or double value is treated as two separate writes: one to each 32-bit half. This can result in a situation where a thread sees the first 32 bits of a 64-bit value from one write, and the second 32 bits from another write.

<https://docs.oracle.com/javase/specs/jls/se11/html/jls-17.html#jls-17.6>



Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = 42;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = 42;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

- Program prints 0
- Explain this using some hardware optimization
 - store buffering, load buffering, invalidate queue, interconnect topology

Visibility

```
class Data { long x; }  
static Data shared;  
void threadA() {  
    Data d = new Data();  
    d.x = 42;  
    shared = d;  
}  
void threadB() {  
    System.out.println(shared.x);  
}
```

- Program prints 0
- Explain this using some compiler optimization

Visibility: trust nobody

Non-synchronized concurrent access to any memory location

- Static field
- Instance field
- Array element

could lead to subtle and hard-to-diagnose bugs.

Question time

Question: Which Java-specific concepts help you to avoid visibility problems?



Visibility: trust friends only

Non-synchronized concurrent access to any memory location

- Static field
- Instance field
- Array element

could lead to subtle and hard-to-diagnose bugs.

Synchronization operations:

- `Thread.start`, `Thread.join`
- `Lock.lock`, `Lock.unlock` and other `java.util.concurrent` primitives
- `synchronized`
- `AtomicLong`, `AtomicReference` read-modify-write operations

Visibility: trust friends only

Non-synchronized concurrent access to any memory location

- Static field
- Instance field
- Array element

could lead to subtle and hard-to-diagnose bugs.

Synchronization operations:

- `Thread.start`, `Thread.join`
- `Lock.lock`, `Lock.unlock` and other `java.util.concurrent` primitives
- `synchronized`
- `AtomicLong`, `AtomicReference` read-modify-write operations

By default³⁵ they act as full barrier

³⁵Consult with javadoc first

Visibility: trust friends only

Non-synchronized concurrent access to any memory location

- Static field
- Instance field
- Array element

could lead to subtle and hard-to-diagnose bugs.

Synchronization operations:

- `Thread.start`, `Thread.join`
- `Lock.lock`, `Lock.unlock` and other `java.util.concurrent` primitives
- `synchronized`
- `AtomicLong`, `AtomicReference` read-modify-write operations

By default³⁵ they act as full barrier and as a linearization point

³⁵Consult with javadoc first

Lecture plan

- 1 Compiler optimizations
- 2 Barriers: kinds and flavours
- 3 Language memory models
 - Goals and non-goals
 - Partial memory model: bad things can not be expressed
 - Partial memory model: bad things should be avoided
 - Strict consistency: GIL
 - Strict consistency: Event Loop
 - Partial memory model: threading as a service
 - Partial memory model: concurrency-aware language subset
 - Memory model: concurrency embedded into language
- 4 Advanced topics
 - Visibility
 - Volatile
- 5 Summary

Using memory location as synchronization point

```
class LockPreReservedByB { boolean busy = true; }
static LockPreReservedByB lock; static Data data;
void threadA() { while (lock.busy) { /*await*/ } // lock.lock()
                Data x = data;
}
void threadB() { Data d = new Data(); d.x = 42;
                data = d;
                lock.busy = false; // lock.unlock()
}
```

Using memory location as synchronization point

```
class LockPreReservedByB { boolean busy = true; }
static LockPreReservedByB lock; static Data data;
void threadA() { while (lock.busy) { /*await*/ } // lock.lock()
                Data x = data;
}
void threadB() { Data d = new Data(); d.x = 42;
                data = d;
                lock.busy = false; // lock.unlock()
}
```

We need

- Lock.unlock to act as publishing memory barrier (store release)
- Lock.lock to repair consistency of data being read (load acquire)
- all operations with lock.busy to be ordered (linear order)

Using memory location as synchronization point

Roach motel idiom³⁶

```
int a; volatile boolean ready = false;
void threadA() {
    a = 41;
    a = 42;
    ready = true; // release data. Every write above will be visible (flushed)
    a = 43;
}
void threadB() {
    while (!ready) {} // acquire data. Every read below will see up-to-date data
    println(a);
}
```

³⁶ <https://shipilev.net/talks/narnia-2555-jmm-pragmatics-en.pdf>

Using memory location as synchronization point

Roach motel idiom³⁶

```
int a; volatile boolean ready = false;
void threadA() {
    a = 41;
    a = 42;
    ready = true; // release data. Every write above will be visible (flushed)
    a = 43;
}
void threadB() {
    while (!ready) {} // acquire data. Every read below will see up-to-date data
    println(a);
}
```

Possible to see 42 or 43. Impossible to see 0 or 41.

³⁶<https://shipilev.net/talks/narnia-2555-jmm-pragmatics-en.pdf>

Volatile

volatile variable

- volatile mode accesses are totally ordered
 - coherence, visibility (progress)
- not a plain variable, enforces some ordering with other locations
 - load acquire, store release
- not an atomic read-modify-write variable, does not provide full barrier semantics

Volatile

volatile variable

- volatile mode accesses are totally ordered
 - coherence, visibility (progress)
- not a plain variable, enforces some ordering with other locations
 - load acquire, store release
- not an atomic read-modify-write variable, does not provide full barrier semantics

Kinda *weak*

- advanced protocols
- performance

Volatile

volatile variable

- volatile mode accesses are totally ordered
 - coherence, visibility (progress)
- not a plain variable, enforces some ordering with other locations
 - load acquire, store release
- not an atomic read-modify-write variable, does not provide full barrier semantics

Kinda *weak*

- advanced protocols
- performance

Very Java-specific

- volatile in C/C++ is **absolutely different**
- `memory_order_acq_rel` in C/C++ is **different** from Java volatile
- many languages are OK with plain variables and full-fence variables
- low-level languages allow to use atomic RMW variables with different "strength"

Volatile: rules of thumb

Volatile: rules of thumb

- use `volatile` only when it simplifies your synchronization policy

Volatile: rules of thumb

- use `volatile` only when it simplifies your synchronization policy
- avoid `volatile` when verifying correctness would require subtle reasoning about visibility

Volatile: rules of thumb

- use `volatile` only when it simplifies your synchronization policy
- avoid `volatile` when verifying correctness would require subtle reasoning about visibility
- good uses of `volatile` include ensuring the visibility of their own state or indicating that an important life-cycle event (such as initialization or shutdown) has occurred

Volatile: rules of thumb

- use `volatile` only when it simplifies your synchronization policy
- avoid `volatile` when verifying correctness would require subtle reasoning about visibility
- good uses of `volatile` include ensuring the visibility of their own state or indicating that an important life-cycle event (such as initialization or shutdown) has occurred

```
volatile boolean asleep;
```

```
...
```

```
while (!asleep) {  
    countSomeSheep();  
}
```

Volatile: rules of thumb

- use `volatile` only when it simplifies your synchronization policy
- avoid `volatile` when verifying correctness would require subtle reasoning about visibility
- good uses of `volatile` include ensuring the visibility of their own state or indicating that an important life-cycle event (such as initialization or shutdown) has occurred

```
volatile boolean asleep;
```

```
...
```

```
while (!asleep) {  
    countSomeSheep();  
}
```

You can use `volatile` variables only when all the following criteria are met:

Volatile: rules of thumb

- use `volatile` only when it simplifies your synchronization policy
- avoid `volatile` when verifying correctness would require subtle reasoning about visibility
- good uses of `volatile` include ensuring the visibility of their own state or indicating that an important life-cycle event (such as initialization or shutdown) has occurred

```
volatile boolean asleep;
```

```
...
```

```
while (!asleep) {  
    countSomeSheep();  
}
```

You can use `volatile` variables only when all the following criteria are met:

- writes to the variable do not depend on its current value, or you can ensure that only a single thread ever updates the value

Volatile: rules of thumb

- use `volatile` only when it simplifies your synchronization policy
- avoid `volatile` when verifying correctness would require subtle reasoning about visibility
- good uses of `volatile` include ensuring the visibility of their own state or indicating that an important life-cycle event (such as initialization or shutdown) has occurred

```
volatile boolean asleep;
```

```
...
```

```
while (!asleep) {  
    countSomeSheep();  
}
```

You can use `volatile` variables only when all the following criteria are met:

- writes to the variable do not depend on its current value, or you can ensure that only a single thread ever updates the value
- the variable does not participate in invariants with other state variables

Volatile: rules of thumb

- use `volatile` only when it simplifies your synchronization policy
- avoid `volatile` when verifying correctness would require subtle reasoning about visibility
- good uses of `volatile` include ensuring the visibility of their own state or indicating that an important life-cycle event (such as initialization or shutdown) has occurred

```
volatile boolean asleep;
```

```
...
```

```
while (!asleep) {  
    countSomeSheep();  
}
```

You can use `volatile` variables only when all the following criteria are met:

- writes to the variable do not depend on its current value, or you can ensure that only a single thread ever updates the value
- the variable does not participate in invariants with other state variables
- locking is not required for any other reason while the variable is being accessed

Summary

Compiler optimizations

- reorder/invent/delete memory operations, prevent it via compiler barriers

Language memory model – bridge between

- chaos of hardware, anarchy of compiler optimizations
- consistency requirements for high-level concurrent algorithms

Useful concepts

- immutability
- DSL for parallel computations
- strict consistency via mutex
- strict consistency via single executor and cooperative multitasking
- threads cannot be implemented as a library
- language memory model is harder to formalize than hardware memory model
- visibility is subtle but critically important
- volatile is for hackers

The most efficient and elegant approach to writing concurrent code

`https://go.dev/ref/mem`

The most efficient and elegant approach to writing concurrent code

`https://go.dev/ref/mem`

Don't be clever